

Министерство образования и науки Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
ИТМО»

ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ
Направление подготовки: 11.03.02 Инфокоммуникационные технологии
и системы связи

Лабораторная работа №5

Выполнил:
Бацанова Екатерина Александровна
Группа К3323
Проверил:
Марченко Елена Вадимовна

Санкт-Петербург
2025

СОДЕРЖАНИЕ

Стр.

Цель	3
Ход работы	4
1 Основная страница.....	4
2 Запуск видео	7
3 Включение/отключение музыки	9
4 Установка напоминания	10
5 Запуск мини-игры с движущимся по экрану шариком	11
6 Переход на вторую страницу сайта	14
7 Открытие почтового клиента с предзаполненным сообще- нием	16
8 Скрытие/показ скрытого текста	16
9 Мини-игра с угадыванием числа	17
10 Показ случайных страниц на Википедии	19
11 Показ текущего расположения пользователя на Яндекс- картах	19
Вывод.....	21

ЦЕЛЬ

Цель работы: создать карту изображений с интерактивными областями, обеспечивающую различные действия при взаимодействии с пользователем.

Задачи:

1. Выбрать изображение для использования в качестве основы для карты;
2. Определить не менее 5 интерактивных областей разной формы (прямоугольник, круг, многоугольник);
3. Настроить действия при клике на каждую область;
4. Использовать HTML-тег `<map>` для создания карты изображений и связать карту с изображением с помощью атрибута `usemap`;
5. Протестировать работоспособность карты в браузере.

ХОД РАБОТЫ

1 Основная страница

В ходе работы была создана интерактивная страница с картой изображений, которая предоставляет пользователю возможность взаимодействовать с различными областями на изображении. Страница включает в себя несколько элементов, таких как изображение, видео, музыка и скрытый текст, а также различные действия при клике на области.

Было выбрано изображение с котиками, к которому прикреплена карта с различными интерактивными областями. Для этого использовался тег `<img>` с атрибутом `usemap`, связывающим изображение с картой.

На изображении определены несколько областей с различными формами (прямоугольник, круг, многоугольник). Каждая область выполняет своё действие при клике (код для всех областей представлен на рис. 1):

1. Запуск видео;
2. Включение/отключение музыки;
3. Установка напоминания;
4. Запуск мини-игры с движущимся по экрану шариком;
5. Переход на вторую страницу сайта;
6. Открытие почтового клиента с предзаполненным сообщением;
7. Скрытие/показ скрытого текста;
8. Мини-игра с угадыванием числа;
9. Показ случайных страниц на Википедии;
10. Показ текущего расположения пользователя на Яндекс-картах.

Далее мы рассмотрим реализацию каждого пункта подробно.

```

1  <!DOCTYPE html>
2  <html lang="ru">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1">
6      <title>Карта изображений</title>
7      <link rel="stylesheet" href="styles.css">
8  </head>
9  <body>
10
11      <h1>Интерактивная карта изображения</h1>
12
13      <div class="image-container">
14          
15
16          <div id="video-container">
17              <div id="white-cover"></div>
18              <video id="overlay-video">
19                  <source src="data/happyhappy.mp4" type="video/mp4">
20              </video>
21          </div>
22
23          <div id="hidden-text">Кот – популярное домашнее животное</div>
24      </div>
25
26      <map id="imgmap" name="imgmap">
27          <area shape="rect" coords="590,331,703,573" href="#"
28              | onclick="playVideo(event, 590, 331, 703, 573)" alt="Видео">
29          <area shape="circle" coords="175,349,52" href="#" id="music-area"
30              | onclick="toggleMusic(event)" alt="Музыка">
31          <audio id="background-music" src="data/meowmeow.mp3"></audio>
32          <area shape="circle" coords="213,447,54" href="#"
33              | onclick="setReminder(event, 213,447,54)" alt="Напоминание">
34          <area shape="poly" coords="23,133,79,55,97,66,113,107,88,109,89,129,123,144,125,165,99,189,44,186,25,154"
35              | href="https://mail.google.com/mail/?view=cm&fs=1&to=kathy.a.b.2004@gmail.com&su=Привет&body=Как_дела?"
36              | target="_blank" alt="Письмо">
37          <area shape="poly" coords="190,661,342,583,493,825,348,889,237,775" href="#"
38              | onclick="startBallMovement()" alt="Рандом">
39          <area shape="circle" coords="571,1001,68" href="back.html" alt="Переключить на новую страницу" />
40          <area shape="rect" coords="8,926,170,1262" href="#"
41              | onclick="toggleText(event, 8, 926, 170, 1262)" alt="Скрытый текст">
42          <area shape="rect" coords="495,58,602,289" href="#" onclick="guessNumber()" alt="Мини-игра">
43          <area shape="rect" coords="602,58,697,289" href="https://ru.wikipedia.org/wiki/Special:Random"
44              | target="_blank" alt="Википедия">
45          <area shape="rect" coords="388,1137,680,1279" target="_blank" alt="Карты" id="map-area">
46      </map>
47
48      <script src="script.js"></script>
49  </body>
50 </html>

```

Рисунок 1 — Код файла *map.html*.

```

1  body {
2      font-family: Arial, sans-serif;
3      text-align: center;
4  }
5
6  .image-container {
7      position: relative;
8      display: inline-block;
9  }
10
11  #main-image {
12      width: 100%;
13      display: block;
14  }
15
16  #video-container {
17      position: absolute;
18      z-index: 10;
19  }
20
21  #hidden-text {
22      position: absolute;
23      background: rgba(255, 142, 142, 0.8);
24      padding: 10px;
25      border: 1px solid black;
26      font-size: 18px;
27      font-weight: bold;
28      text-align: center;
29      z-index: 10;
30      transform: rotate(-35deg);
31  }

```

Рисунок 2 — Код файла *styles.css* со стилями главной страницы.

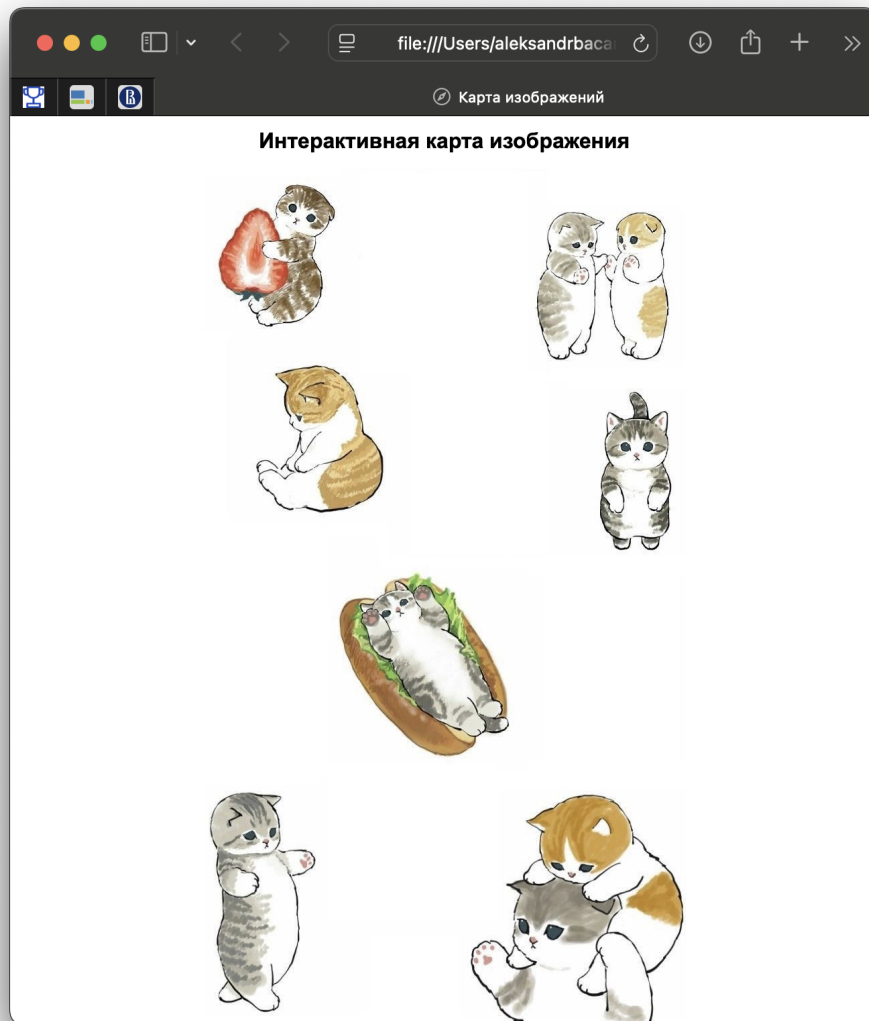


Рисунок 3 — Вид страницы *map.html* в браузере.

2 Запуск видео

Функция `playVideo` (рис. 5) запускает видео в выбранной области изображения при клике. Она:

- Предотвращает стандартное действие клика (при клике на элемент иногда происходит не только сам клик, но и возможное выполнение какого-то стандартного действия браузера, в моем случае – перенос к началу страницы);

- Рассчитывает масштаб изображения, чтобы правильно позиционировать видео в зависимости от размера изображения;
- Определяет размеры и позицию видео, устанавливая их с учетом масштабирования;
- Показывает белое покрытие, чтобы закрыть часть изображения, на которую накладывается видео;
- Воспроизводит видео;
- После завершения видео скрывает контейнер с видео.

Таким образом, видео появляется в нужной области изображения и исчезает после окончания.

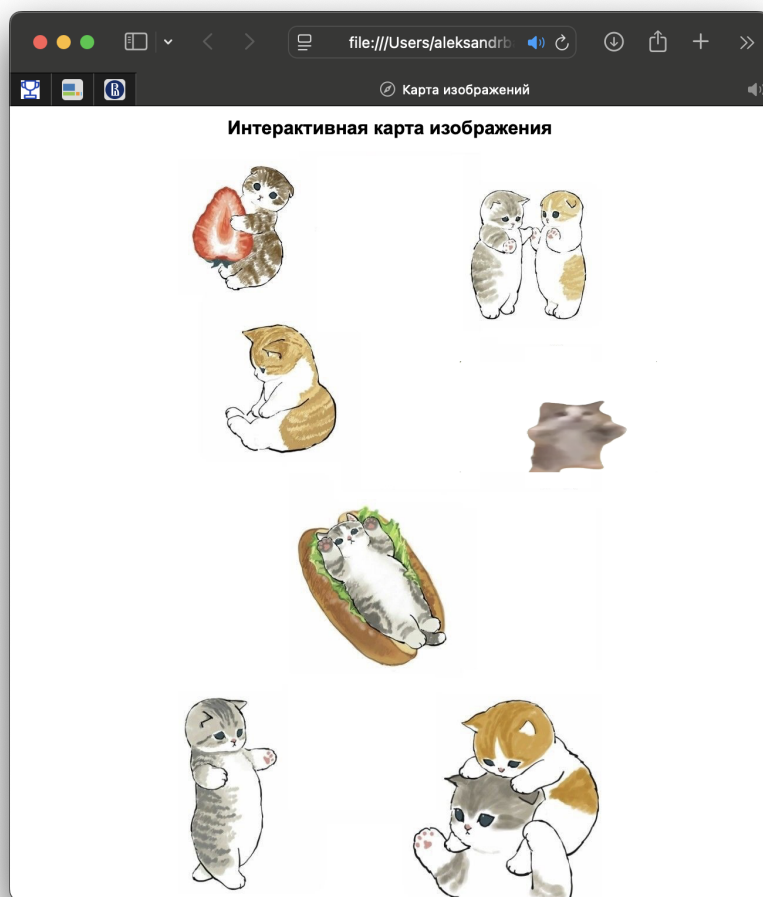


Рисунок 4 — Вид страницы *map.html*, когда включено видео.

```

1  document.addEventListener("DOMContentLoaded", function () {
2      document.getElementById("video-container").style.display = "none";
3      document.getElementById("hidden-text").style.display = "none";
4  });
5
6  function playVideo(event, x1, y1, x2, y2) {
7      event.preventDefault();
8
9      const videoContainer = document.getElementById("video-container");
10     const whiteCover = document.getElementById("white-cover");
11     const video = document.getElementById("overlay-video");
12     const image = document.getElementById("main-image");
13
14     const scaleX = image.clientWidth / image.naturalWidth;
15     const scaleY = image.clientHeight / image.naturalHeight;
16
17     const width = (x2 - x1) * scaleX;
18     const height = (y2 - y1) * scaleY;
19     const left = x1 * scaleX;
20     const top = y1 * scaleY;
21
22     videoContainer.style.width = `${width}px`;
23     videoContainer.style.height = `${height}px`;
24     videoContainer.style.top = `${top}px`;
25     videoContainer.style.left = `${left}px`;
26     videoContainer.style.display = "block";
27
28     whiteCover.style.width = `${width}px`;
29     whiteCover.style.height = `${height}px`;
30     whiteCover.style.position = "absolute";
31     whiteCover.style.top = "0";
32     whiteCover.style.left = "0";
33     whiteCover.style.background = "white";
34     whiteCover.style.opacity = "1";
35
36     video.style.width = `${width * 3}px`;
37     video.style.height = `${height * 3}px`;
38     video.style.position = "absolute";
39     video.style.top = "50%";
40     video.style.left = "50%";
41     video.style.transform = "translate(-50%, -50%)";
42     video.style.zIndex = "11";
43
44     video.play();
45
46     video.onended = function () {
47         videoContainer.style.display = "none";
48     };
49 }

```

Рисунок 5 — Код функции playVideo в файле *script.js*.

3 Включение/отключение музыки

Функция toggleMusic (рис. 7) управляет воспроизведением музыки. При клике:

- Если музыка играет, она останавливается и начинается с начала;
- Если музыка не играет, она запускается.

Флаг `isMusicPlaying` отслеживает, воспроизводится ли музыка в данный момент.

```
52 let isMusicPlaying = false;
53 let audio = new Audio('meowmeow.mp3');
54
55 function toggleMusic(event) {
56     if (event) event.preventDefault();
57     if (isMusicPlaying) {
58         audio.pause();
59         audio.currentTime = 0;
60     } else {
61         audio.play();
62     }
63     isMusicPlaying = !isMusicPlaying;
64 }
```

Рисунок 6 — Код функции `toggleMusic` в файле *script.js*.

4 Установка напоминания

Функция `setReminder` (рис. ??) устанавливает напоминание:

- При клике появляется первое напоминание в виде `alert` с текстом "Не забудьте о важном событии!";
- Через 5 секунд (5000 миллисекунд) появляется второе напоминание с текстом "Прошло 5 секунд! Напоминаем еще раз."

Метод `event.preventDefault()` предотвращает стандартное поведение браузера – отбрасывание к началу страницы.

```

52 let isMusicPlaying = false;
53 let audio = new Audio('meowmeow.mp3');
54
55 function toggleMusic(event) {
56     if (event) event.preventDefault();
57     if (isMusicPlaying) {
58         audio.pause();
59         audio.currentTime = 0;
60     } else {
61         audio.play();
62     }
63     isMusicPlaying = !isMusicPlaying;
64 }

```

Рисунок 7 — Код функции setReminder в файле *script.js*.

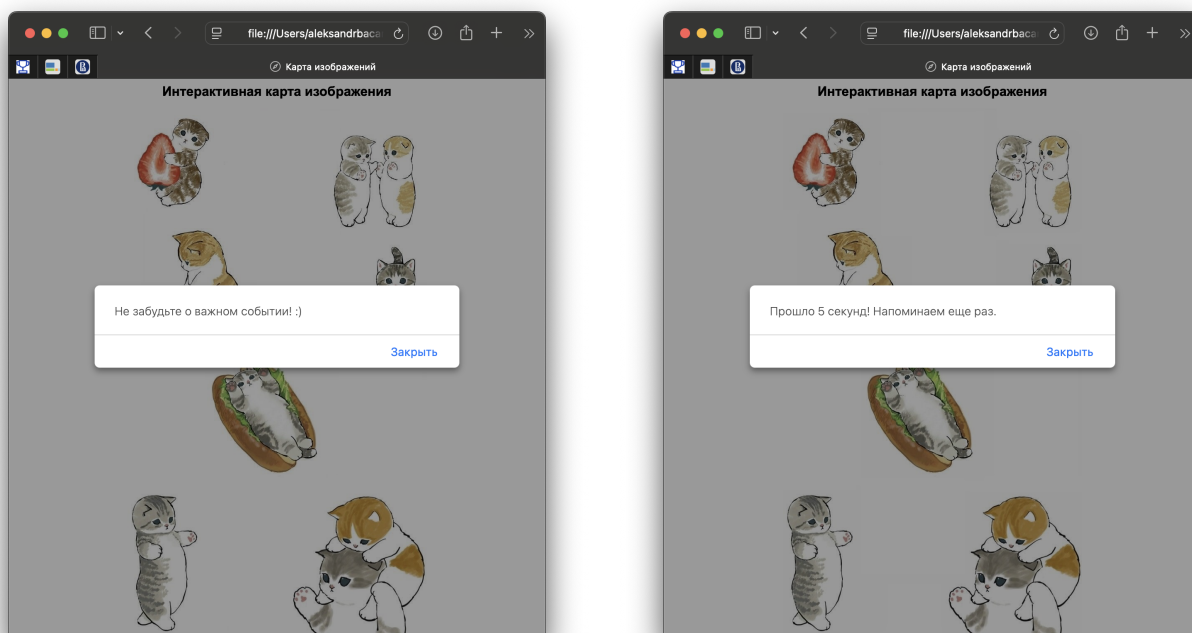


Рисунок 8 — Вывод уведомлений-напоминаний.

5 Запуск мини-игры с движущимся по экрану шариком

Этот код реализует игру, в которой нужно убежать от движущегося шарика

- Хранение состояния игры:

Когда игра загружается, функция `loadGameState` проверяет, есть ли сохранённое состояние игры в `localStorage`. Если данные найдены, восстанавливаются координаты шарика и его скорость. Если данных нет, игра начинается с исходных значений.

В процессе игры состояние сохраняется с помощью функции `saveGameState`, которая записывает текущие данные (позиция шарика, его скорость и т.д.) в `localStorage`.

- Движение шарика:

Когда пользователь двигает мышь, координаты мыши сохраняются в переменных `mouseX` и `mouseY`. Шарик двигается в сторону курсора с расчётом скорости и направления, и его позиция обновляется каждый раз в функции `moveBall`. Для этого используется формула для нахождения направления от шарика к мыши.

- Обработка столкновения:

Когда шарик оказывается достаточно близко к курсору (попадает в радиус 25 пикселей), игра заканчивается. Шарик исчезает, появляется сообщение о проигрыше, и состояние игры сбрасывается.

- Старт игры:

Игра начинается при клике на область карты с атрибутом `alt="Рандом"`. В этот момент создаётся элемент шарика, и начинается его движение.

- Увеличение скорости:

Каждую секунду увеличивается скорость шарика. Это реализовано через `setInterval`, который раз в секунду увеличивает значение `speed` и сохраняет обновлённое состояние.

- Сохранение и сброс состояния:

Если игрок проиграл, все данные игры удаляются из `localStorage`, что позволяет начать игру заново при следующем заходе.

```

115 let ball;
116 let ballX, ballY;
117 let mouseX, mouseY;
118 let speed = 1;
119 let ballSpeedX = 0;
120 let ballSpeedY = 0;
121 let interval;
122 let isBallMoving = false;
123 let isGameActive = false;
124
125 function loadGameState() {
126   if (localStorage.getItem("ballX")) {
127     ballX = parseFloat(localStorage.getItem("ballX"));
128     ballY = parseFloat(localStorage.getItem("ballY"));
129     speed = parseFloat(localStorage.getItem("speed"));
130     ballSpeedX = parseFloat(localStorage.getItem("ballSpeedX"));
131     ballSpeedY = parseFloat(localStorage.getItem("ballSpeedY"));
132     isBallMoving = true;
133   } else {
134     resetBall();
135   }
136 }
137
138 function saveGameState() {
139   localStorage.setItem("ballX", ballX);
140   localStorage.setItem("ballY", ballY);
141   localStorage.setItem("speed", speed);
142   localStorage.setItem("ballSpeedX", ballSpeedX);
143   localStorage.setItem("ballSpeedY", ballSpeedY);
144 }
145
146 function resetBall() {
147   ballX = 0;
148   ballY = 0;
149   speed = 1;
150   ballSpeedX = 0;
151   ballSpeedY = 0;
152 }
153
154 setInterval(() => {
155   if (isBallMoving) {
156     speed += 1.5;
157     saveGameState();
158   }
159 }, 1000);
160
161 document.addEventListener("mousemove", function(event) {
162   mouseX = event.pageX;
163   mouseY = event.pageY;
164 });

```

```

166 function moveBall() {
167   let deltaX = mouseX - ballX;
168   let deltaY = mouseY - ballY;
169   let distance = Math.sqrt(deltaX * deltaX + deltaY * deltaY);
170
171   if (distance > 1) {
172     ballSpeedX = deltaX / distance * speed;
173     ballSpeedY = deltaY / distance * speed;
174
175     ballX += ballSpeedX;
176     ballY += ballSpeedY;
177
178     ball.style.position = "absolute";
179     ball.style.left = ballX - 25 + "px";
180     ball.style.top = ballY - 25 + "px";
181     ball.style.width = "50px";
182     ball.style.height = "50px";
183     ball.style.backgroundColor = "#00FF00";
184     ball.style.border = "3px solid black";
185     ball.style.borderRadius = "50%";
186   }
187
188   if (Math.abs(mouseX - ballX) < 25 && Math.abs(mouseY - ballY) < 25) {
189     ball.remove();
190     alert("Ву поваргани!");
191     clearInterval(interval);
192     isBallMoving = false;
193     localStorage.removeItem("ballX");
194     localStorage.removeItem("ballY");
195     localStorage.removeItem("speed");
196     localStorage.removeItem("ballSpeedX");
197     localStorage.removeItem("ballSpeedY");
198     isGameActive = false;
199   }
200 }
201
202 function startBallMovement() {
203   if (isGameActive) {
204     return;
205   }
206
207   isGameActive = true;
208   resetBall();
209   ball = document.createElement("div");
210   document.body.appendChild(ball);
211
212   alert("Убери от шарика!");
213
214   isBallMoving = true;
215   interval = setInterval(moveBall, 10);
216 }
217
218 document.querySelector('area[alt="Рандом"]').addEventListener('click', startBallMovement);
219 window.onload = loadGameState;
220
221 function updateMapLink(latitude, longitude) {
222   const mapLink = document.querySelector("#map-area");
223   const url = "https://yandex.ru/maps/?ll=${longitude},${latitude}&z=17";
224   mapLink.href = url;
225 }

```

Рисунок 9 — Код для реализации мини-игры с движущимся по экрану шариком.

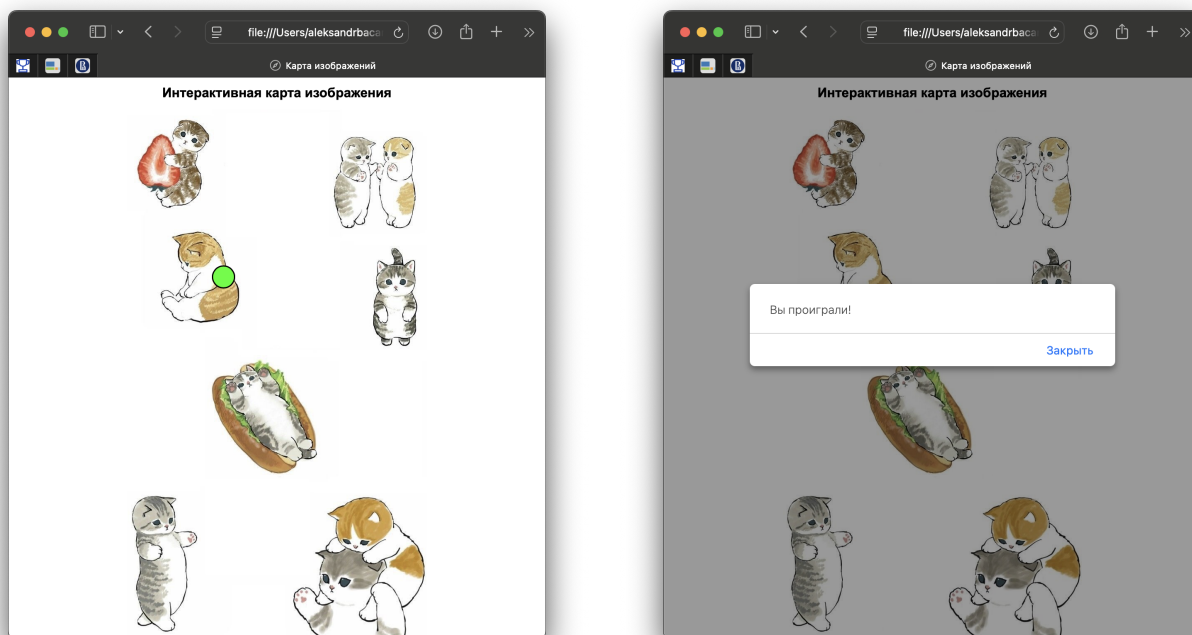


Рисунок 10 — Пример работы мини-игры с движущимся по экрану шариком.

6 Переход на вторую страницу сайта

Код второй страницы сайта представлен ниже.

```

1  <!DOCTYPE html>
2  <html lang="ru">
3  <head>
4  |   <meta charset="UTF-8">
5  |   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  |   <title>Страница с кнопкой возврата</title>
7  |   <link rel="stylesheet" href="styles_back.css">
8  </head>
9  <body>
10 |   <h1>Добро пожаловать на страницу!</h1>
11 |   <p>Чтобы вернуться на главную, нажмите на картинку ниже.</p>
12 |   <a href="map.html" class="btn">
13 |   |   
14 |   </a>
15 </body>
16 </html>

```

Рисунок 11 — Код файла *back.html*.

```

1  body {
2      font-family: Arial, sans-serif;
3      text-align: center;
4      padding: 50px;
5  }
6  .btn {
7      background-color: transparent;
8      border: none;
9      cursor: pointer;
10 }
11 .btn img {
12     width: 120px;
13     height: auto;
14 }
15 .btn:hover img {
16     opacity: 0.8;
17 }

```

Рисунок 12 — Код файла *back.html* со стилями второй страницы.

При клике на область мы попадаем на эту страничку с возможностью вернуться.

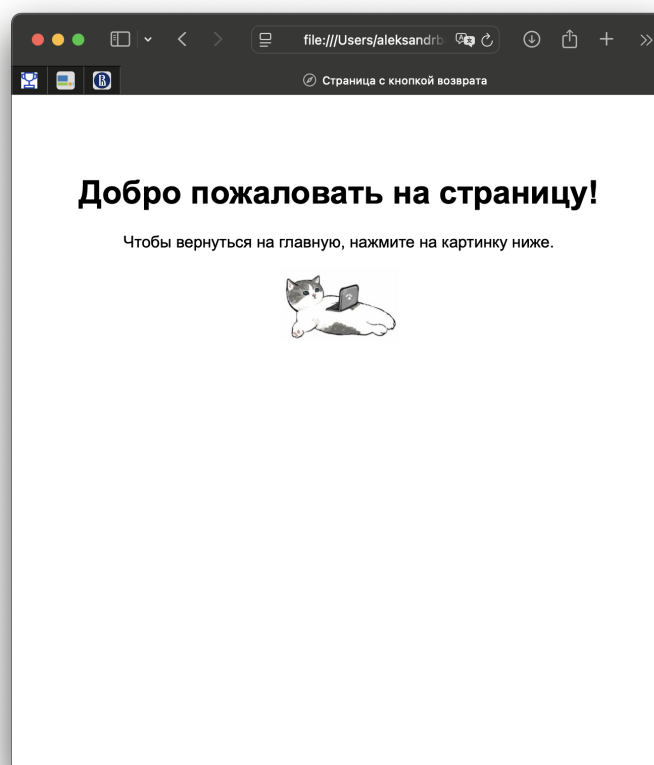


Рисунок 13 — Вид страницы *back.html* в браузере.

7 Открытие почтового клиента с предзаполненным сообщением

Эта часть выполнена непосредственно в HTML. При нажатии на область открывается gmail с предзаполненным письмом

- Адрес электронной почты – мой;
- Тема – «Привет»;
- Содержание – «Как дела?».

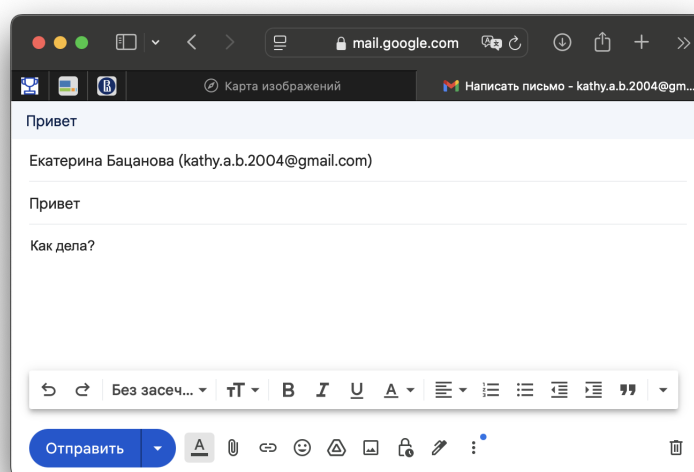


Рисунок 14 — Открытие предзаполненного письма в gmail через *map.html*.

8 Соккрытие/показ скрытого текста

Функция `toggleText` (рис. 15) отображает или скрывает текст на изображении при клике.

- Рассчитываются размеры и положение текста с учётом масштабирования изображения;
- Текст появляется в заданной области, если он скрыт, или скрывается, если он уже виден.

```

75 function toggleText(event, x1, y1, x2, y2) {
76     event.preventDefault();
77     const textElement = document.getElementById("hidden-text");
78     const image = document.getElementById("main-image");
79
80     const scaleX = image.clientWidth / image.naturalWidth;
81     const scaleY = image.clientHeight / image.naturalHeight;
82
83     const width = (x2 - x1) * scaleX;
84     const height = (y2 - y1) * scaleY;
85     const left = (x1 + 70) * scaleX;
86     const top = (y1 + 80) * scaleY;
87
88     textElement.style.width = `${width}px`;
89     textElement.style.height = `${height / 5}px`;
90     textElement.style.top = `${top}px`;
91     textElement.style.left = `${left}px`;
92
93     if (textElement.style.display === "none") {
94         textElement.style.display = "block";
95     } else {
96         textElement.style.display = "none";
97     }
98 }

```

Рисунок 15 — Код функции toggleText в файле *script.js*.

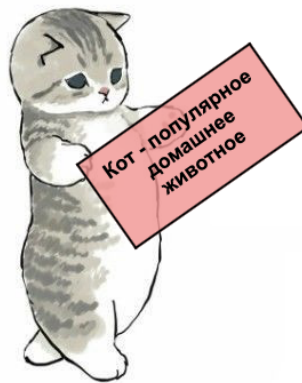


Рисунок 16 — Вид части страницы *map.html* после появления надписи.

9 Мини-игра с угадыванием числа

Функция guessNumber (рис. 17) реализует игру "Угадай число":

- Генерируется случайное число от 1 до 10;
- Пользователь вводит свой вариант через prompt;
- Если пользователь нажимает «Отмена», функция завершится;

- Если введённое число совпадает с загаданным, появляется сообщение «Правильно!», иначе — «Неверно» с правильным ответом.

Это простая игра на угадывание числа.

```
100 function guessNumber() {  
101     const number = Math.floor(Math.random() * 10) + 1;  
102     const guess = prompt("Угадайте число от 1 до 10:");  
103  
104     if (guess === null) {  
105         return;  
106     }  
107  
108     if (parseInt(guess) === number) {  
109         alert("Правильно!");  
110     } else {  
111         alert(`Неверно, правильный ответ: ${number}`);  
112     }  
113 }
```

Рисунок 17 — Вид части страницы *map.html* после появления надписи.

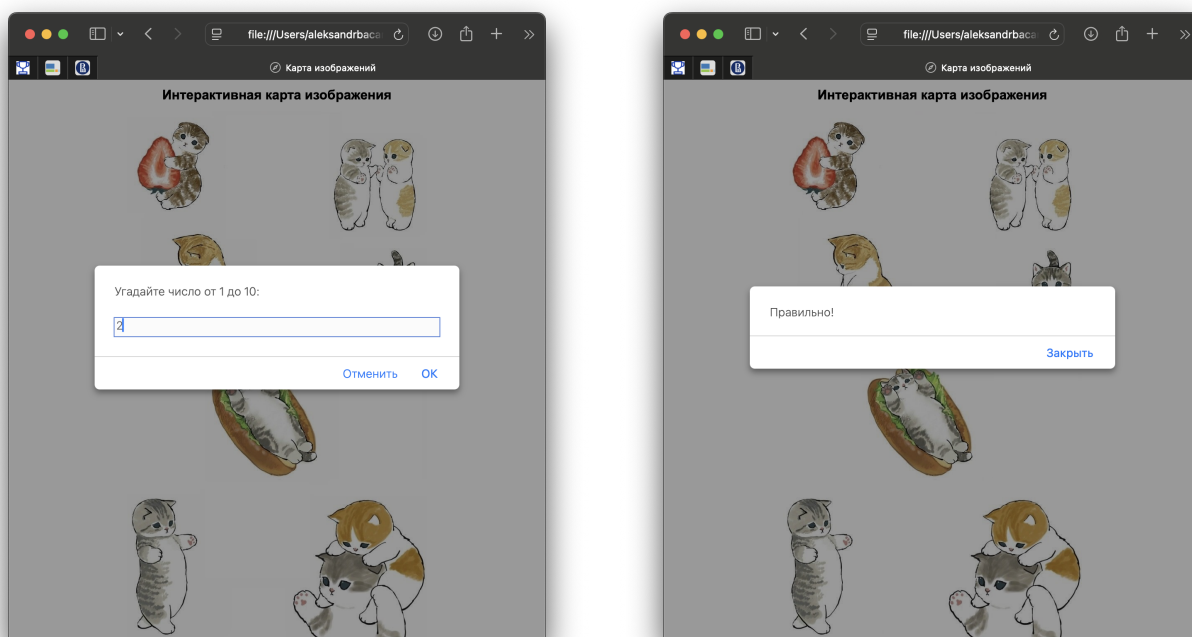


Рисунок 18 — Пример работы мини-игры с угадыванием числа.

10 Показ случайных страниц на Википедии

Это действие также реализовано средствами html. При нажатии на область откроется случайная страница на Википедии (ссылка ведёт на <https://ru.wikipedia.org/wiki/Special:Random>).



Рисунок 19 — Открытая через *map.html* случайная страница на Википедии.

11 Показ текущего расположения пользователя на Яндекс-картах

При клике на данную область определяется местоположение пользователя и открываются Яндекс.Карты с отображением его текущих координат в новой вкладке. Для получения текущих координаты пользователя (широта и долгота) используется API `navigator.geolocation.getCurrentPosition`.

```

221 function updateMapLink(latitude, longitude) {
222     const mapLink = document.querySelector('#map-area');
223     const url = `https://yandex.ru/maps/?ll=${longitude},${latitude}&z=17`;
224     mapLink.href = url;
225 }
226
227 document.querySelector('#map-area').addEventListener('click', function(event) {
228     event.preventDefault();
229
230     navigator.geolocation.getCurrentPosition(function(position) {
231         const latitude = position.coords.latitude;
232         const longitude = position.coords.longitude;
233
234         updateMapLink(latitude, longitude);
235         window.open(document.querySelector('#map-area').href, '_blank');
236     });
237 });

```

Рисунок 20 — Код метода для показа текущего местоположения пользователя.

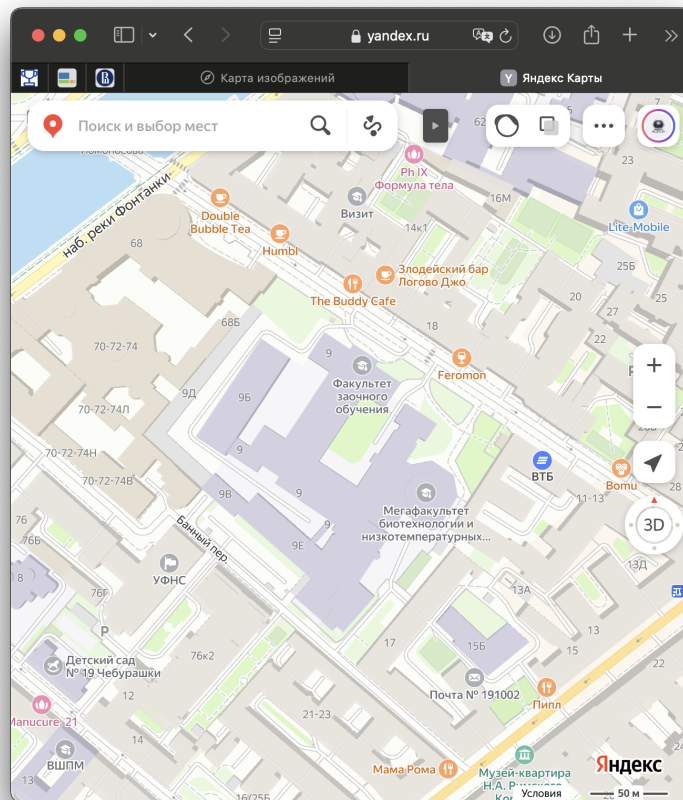


Рисунок 21 — Пример работы метода для показа текущего местоположения пользователя.

ВЫВОД

В ходе работы была создана интерактивная карта с областями различной формы, каждая из которых выполняет определённое действие при взаимодействии с пользователем. Используются функции для воспроизведения видео, воспроизведения музыки, показа уведомлений, игр и геолокации. Также реализована интеграция с Яндекс.Картами для отображения текущего местоположения пользователя. Всё это позволяет создать динамичную страницу.